

MACE: A Multicore Agentic Co-Design Engine

Anonymous Authors

CHIA A3 Hackathon Submission, September 2026

Abstract—MACE is an agentic AI system that plans, builds, and verifies multicore OpenPiton configurations on the CHIA framework. Phase 1 adds `chia_openpiton`, a CHIA adapter for OpenPiton’s configure, build, run, and collect operations, with 228 tests that need no OpenPiton checkout. Phase 2 adds the MACE loop: a planner breaks a hardware objective into tasks, the tasks run in parallel, and a deterministic integrator gates every change on Verilator simulation. A failure-analysis agent diagnoses failures, and the planner re-plans around them.

We tested MACE against two baselines, manual mesh scaling and a one-shot LLM prompt, on 2x2 and 4x4 meshes, with every result gated by Verilator simulation of the RTL. Besides OpenPiton’s Ariane and OpenSPARC T1 cores, we brought up PicoRV32, fixing two bugs in its OpenPiton integration and enabling an existing SRAM define. We also traced a Verilator 5 scheduling bug that makes CVA6’s cache adapter drop invalidations, and worked around it with atomic barrier polling. With these changes, MACE’s loop passes 2x2 and 4x4 meshes of both Ariane and PicoRV32, while the one-shot baseline fails every configuration it attempts.

1. Introduction

Current agentic coding systems are typically benchmarked on isolated tasks or toy problems [1], [2], which misses the complexity of hardware construction. Bringing up a multicore system means navigating a large RTL codebase (OpenPiton [3] spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a fragile toolchain (version skew, cross-compilation bugs, checkout-specific issues), and verification that takes minutes to hours per attempt.

MACE is designed for this environment. Given a hardware objective, such as “bring up a 2x2 mesh” or “verify workload W passes,” MACE plans tasks, builds and simulates each configuration, diagnoses failures from Verilator [4] output, and iterates. Every validation in MACE is gated by Verilator simulation.

1.1. Features

MACE contributes:

- `chia_openpiton` (Section 3): a CHIA adapter for OpenPiton with a configure/build/run/collect interface, following the structure of CHIA’s ESP adapter.

- The MACE loop (Section 4): planning, parallel task execution, deterministic integration, and LLM-driven failure triage, tested end to end against full Verilator runs, with cost, time, and task metrics.
- PicoRV32 support (Section 5), next to Ariane and OpenSPARC T1, from two fixes to its upstream OpenPiton integration and one existing define.
- An interactive CLI and Verilator code coverage as a second verification axis (Section 6).

2. Background and Related Work

CHIA [5] (release 1.0.1) provides the agent, tool, and dispatch layer this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and Ray-based [6] remote dispatch and caching. OpenPiton [3] is Princeton’s open-source, silicon-proven manycore research framework; Figure 2 shows a mesh and one of its tiles. Its three tile cores, the latter two added through BYOC [7], are OpenSPARC T1 (`sparc`), Ariane/CVA6 [8] (`ariane`), and PicoRV32 [9] (`pico`). Before this project, CHIA had no OpenPiton adapter.

Related work: LLM-based hardware design has mostly been evaluated at module scale. VerilogEval [2] scores single-module Verilog generation, ChipNeMo [10] adapts LLMs to chip-design tasks, and AutoChip [11], RTL-Fixer [12], VerilogCoder [13], and MAGE [14] feed compiler or simulation results back to one or more agents to repair generated RTL. In software, SWE-bench [15] moved evaluation from isolated functions [1] to repository-level issues. MACE applies the same agent-with-feedback idea at system scale: it plans and verifies bring-up of an existing manycore, gated by whole-system Verilator simulation.

3. Phase 1: The OpenPiton Adapter

Design. Figure 3 shows where the adapter sits. `OpenPitonWorkspaceNode` is a CHIA `CollocatedNode` whose resource-pinned remote functions configure, build, run, and collect results. OpenPiton writes generated `.tmp.v` files into its source tree on every build, so each worker gets its own checkout, reserved through one `openpiton` resource unit. Pure parsers read verdicts from OpenPiton’s transcripts, using the strings its RTL monitors emit.

Results. The adapter’s 228 tests that need no OpenPiton checkout all pass, and so do acceptance tests with full Verilator builds, including two checkouts building in parallel:

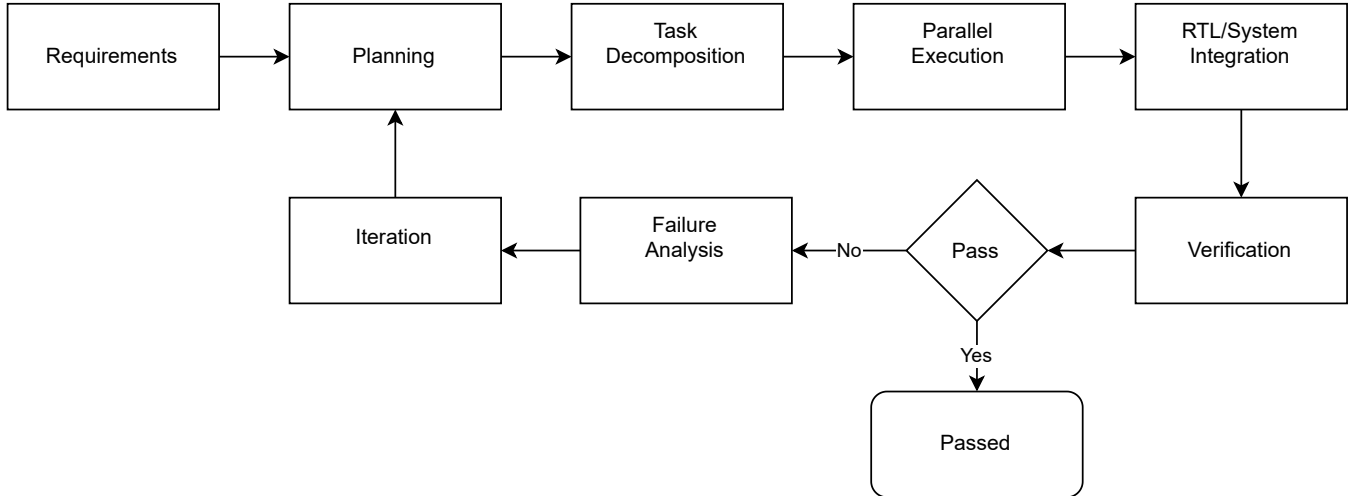


Figure 1. The MACE pipeline. Verification gates every integrated change against Verilator simulation. The run ends when every task passes its gate workload; any failure triggers failure analysis and a replan.

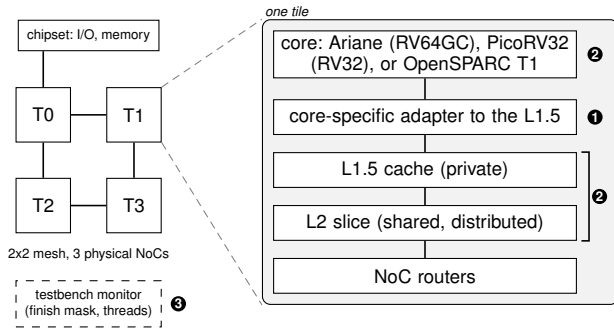


Figure 2. An OpenPiton mesh and one of its tiles. Each core reaches the tile’s L1.5 through its own adapter. The marks show the issues this work fixed or worked around: ① CVA6’s L1.5 adapter dropping invalidations under Verilator 5 (worked around), ② PicoRV32’s self-boot gate and the SRAM model’s power-on self-clear, and ③ the testbench monitor’s 32-bit finish mask and its PicoRV32 thread tracking.

176s versus 324s serial. One acceptance test, 2x2 Ariane on a GCP worker, remains blocked by unreliable dispatch to that worker (Section 9.1).

4. Phase 2: The MACE Loop

Pipeline. Figure 1 shows the loop. A planning agent turns a *MaceSpec* (core, target mesh, gate workloads, objective, budget) into a task DAG, and workers dispatch ready tasks in parallel through Ray, each pinned to its own checkout (Figure 3). A deterministic integrator runs the tasks in dependency order: each builds its own configuration, including any cache or RTL-define overrides the planner gave it, and runs the first gate workload, whose *verdict*, parsed from simulator output, must read *pass*. On a failure, a failure-analysis agent with read-only tools (log grep, file collection, a fixture diff, and an *objdump* symbol check) diagnoses the transcript, and the planner replans around its proposed fix.

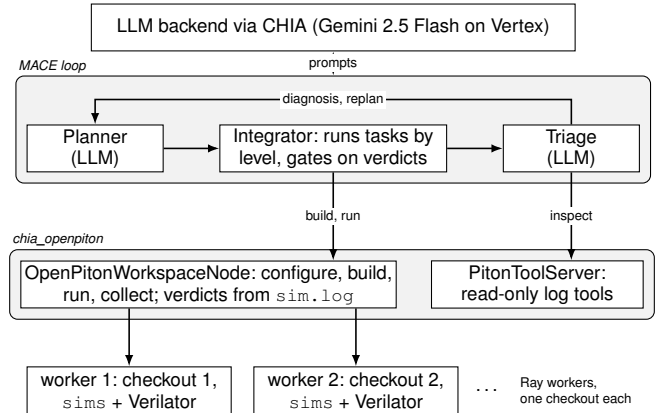


Figure 3. How the pieces fit. The MACE loop drives OpenPiton through *chia_openpiton*, and CHIA and Ray place each checkout’s calls on its own worker.

Gate workloads surface a memory-visibility bug. Writing the gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) showed that a plain *volatile* read of a shared value was not reliably visible from the same hart shortly after the write, while an atomic fetch-and-add-zero read always was. Every gate workload now reads shared values through this *atomic_read()* helper.

Resource enforcement. MACE checks all three Budget dimensions (iterations, USD, wall-clock) before each iteration starts. The USD check reads zero on Vertex, because CHIA’s Vertex backend reports no price (Section 8). MACE verifies the checksums of its C gate programs before any LLM call or checkout access.

TABLE 1. VERILATOR VERDICT AND END-TO-END WALL TIME PER APPROACH, ONE RUN PER CELL. ALL RUNS SHARE ONE BUILD CACHE, SO AN APPROACH PAYS FOR A BUILD ONLY WHEN IT PICKS A NEW CONFIGURATION; (A) COUNTS MACHINE TIME ONLY, EXCLUDING THE TIME WE SPENT FINDING THE FIXES. [†]INVALID CONFIGURATION (L1.5 SIZE 0), REJECTED BEFORE ANY BUILD.

Core	Mesh	(a) Manual	(b) One-shot	(c) MACE
ariane	2x2	pass, 33.9s	fail, 10.4s [†]	pass, 750.1s
	4x4	pass, 271.9s	fail, 3820.9s	pass, 250.9s
pico	2x2	pass, 9.8s	fail, 12.0s [†]	pass, 70.7s
	4x4	pass, 53.9s	fail, 13.4s [†]	pass, 100.6s

5. Adding PicoRV32

Each core reaches its tile’s L1.5 through its own adapter (Figure 2), and a core with its own coherent cache, as Ariane has, needs new coherence-adapter RTL. PicoRV32 has no such cache and already has an upstream transducer, which OpenPiton’s CI builds under Verilator but never runs.

Our first pico runs built but never reached a trap. Tracing them found two bugs and one setting (❶, ❷): the core’s self-boot gate, the monitor’s tracking of pico’s tile, and an SRAM power-on self-clear that discarded pico’s first memory writes, which an existing OpenPiton define turns off. With these changes, PicoRV32 passes under Verilator on 2x2 and 4x4 meshes, every tile reaching `Hit Good trap` on `addi.S`. Its OpenPiton integration has no C compiler, so it runs assembly workloads only.

6. Additional Capabilities

MACE has an EDA-style interactive shell, modeled on Yosys and OpenROAD, that checks core compatibility before a run. For a second verification axis, we patched the OpenPiton testbench to call Verilator’s coverage-write API: a 2x2 Ariane run covers 9497 of 26900 lines (35.3%). The loop reaches its LLM through CHIA’s backends; every result in this paper used Vertex with Gemini 2.5 Flash.

7. Evaluation

We compare three approaches on 2x2 and 4x4 meshes: (a) manual mesh scaling with a time log, (b) a single LLM prompt for a configuration, built and run once with no tools and no retry, and (c) the full MACE loop, with every LLM call on Gemini 2.5 Flash (Table 1). Ariane runs `barrier_atomic.c`; pico runs `addi.S`, since its integration has no C toolchain; SPARC does not yet run under our Verilator (Section 9.1). All three approaches get pico’s `CONFIG_DISABLE_BIST_CLEAR` define: (a) sets it by hand, and the objectives for (b) and (c) name it; (b)’s config line accepts RTL defines, and (c)’s planner requests one through a new `CONFIG_RTL`: directive, which adds it on top of the mesh’s defaults.

(b) fails every configuration. In three of four, `PitonConfig`’s construction-time validation rejected the LLM’s config before any build: it set the L1.5 size to zero,

which every OpenPiton tile needs, and (b) has no retry path. On pico it included the named define both times. At ariane 4x4 it chose oversized caches, including a 4 MB L2 per tile: the build took 35 minutes and the simulation timed out with no tile finished. The two prompts differ: the planner’s lists the default cache sizes and requires positive values, and the planner falls back to the defaults on a malformed cache line. The one-shot prompt has neither.

Mesh scaling to 2x2 and 4x4. Manual scaling (a) first stalled at 2x2 and never reached 4x4, for two reasons. Under Verilator 5, a known scheduling bug [16] makes CVA6’s L1.5 adapter drop cache invalidations (❶ in Figure 2), so the plain-load exit barrier in `syscalls.c` never sees other tiles’ updates; OpenPiton pins a CVA6 that predates the upstream workaround [17]. A hand port of that workaround, which the artifact does not include, passes 2x2 with the unmodified barrier (4 of 4 tiles, 24s), while the original adapter times out with one tile done. Separately, the monitor’s 32-bit `finish_mask` (❷) capped verification at 8 tiles. Our runs keep the pinned adapter, poll the barrier with atomic reads, and widen the mask. With both in place, manual runs pass both sizes, and the loop planned and verified both meshes, every tile confirmed, with no changes to its planning or verification logic (Table 1): 2x2 in 750.1s across four tasks, including an L1D-associativity variant the planner chose to verify, and 4x4 in 250.9s on a cached build. On pico it passes in a single task at both sizes, the 4x4 run also on a cached build.

Recovery. No run above needed a replan, so we broke the loop’s first plan and let it continue, at 2x2 with at most three iterations. Adding `PITON_FPGA_SYNTH`, an FPGA-synthesis define, fails the ariane build, and removing `CONFIG_DISABLE_BIST_CLEAR` times out the pico simulation. With the Table 1 objectives, the loop recovered in 4 of 6 ariane runs and 5 of 6 pico runs, each in its first replan. One run ended at `planning_failed`, and two crashed on an LLM reply cut off at its output limit, which the loop now survives. A replan starts from the objective and the triage feedback and never sees the broken plan, so these runs exercise the cycle more than the diagnosis: triage named the seeded define in 2 of its 8 diagnoses of the build failure and never named the BIST define. With a pico objective that names no define, the loop recovered in none of 3 runs, as every later simulation timed out on the missing BIST define.

8. Cost

All experiments behind this paper cost \$37 of a \$300 GCP credit allocation, covering simulation compute and LLM inference; development used a \$20/month Claude subscription, outside that budget. We built MACE and drafted this paper and its documentation with Claude Code, and we reviewed all text and results. A single loop run is cheap. At 2x2, an ariane run costs about \$0.12 (\$0.04 of LLM tokens and \$0.08 of compute) and a pico run about \$0.02, while a sparc run that exhausts its three-iteration budget would cost an estimated \$0.25, mostly for its one-time build. We estimate compute as `e2-highmem-8` time (\$0.36/hour) over

each run’s wall time, and tokens at Gemini 2.5 Flash’s \$0.30 and \$2.50 per million input and output tokens. MACE’s own cost counter reads zero because CHIA’s Vertex backend reports no price and omits thinking tokens, more than half of the billed output here, so we measured tokens by re-sending representative planner and task prompts.

9. Limitations and Future Directions

9.1. Limitations

OpenSPARC T1 builds but does not run under our Verilator 5.020: the core never receives its power-on wake-up interrupt. OpenPiton’s own CI recipe stalls the same way here, while CI uses Verilator 4, so the cause is environmental; it remains open. Dispatch to a tailnet-relayed GCP worker is also unreliable: exporting the proxy variables `chia up` sets fixed a first hang, but a later attempt hung again with no outbound traffic, pointing at Ray’s scheduling under a proxied topology.

In the reported runs, the LLM decides the task list, cache sizes, and RTL defines; task agents get no tools, and we wrote every RTL and testbench fix in this paper. Each Table 1 cell is a single run on one checkout. The gate programs check shared data on hart 0 only; every other hart must still reach its good trap. The Nix flake pins Verilator 5.052, while the reported runs used 5.020.

9.2. Future Directions

All three cores already had upstream adapters to the L1.5; a core without one, such as Ibex or Rocket Chip, needs it written first. The next step is a reusable adapter template built from the shape the three share (a per-core RTL directory, a transducer, and a core-select arm in the tile template), with MACE’s loop verifying each revision of the adapter RTL in Verilator as an engineer writes it. A dashboard over the run database would show dispatch, build, and verification live.

10. Conclusion

MACE shows that an agentic loop can plan, build, and verify multicore hardware, with every result gated by RTL

simulation. Along the way we fixed RTL and testbench bugs and added PicoRV32 next to Ariane and OpenSPARC T1, and the loop then passed 2x2 and 4x4 meshes of both Ariane and PicoRV32. The next steps are a reusable adapter template for new cores and failure analysis that finds a fix the objective does not name.

References

- [1] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [2] M. Liu *et al.*, “VerilogEval: Evaluating large language models for Verilog code generation,” in *Proc. ICCAD*, 2023.
- [3] J. Balkind *et al.*, “OpenPiton: An open source manycore research framework,” in *Proc. ASPLOS*, 2016, pp. 217–232.
- [4] W. Snyder *et al.*, “Verilator,” <https://verilator.org>.
- [5] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” arXiv:2606.27350, 2026.
- [6] P. Moritz *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. OSDI*, 2018, pp. 561–577.
- [7] J. Balkind *et al.*, “BYOC: A ‘Bring Your Own Core’ framework for heterogeneous-ISA research,” in *Proc. ASPLOS*, 2020, pp. 699–714.
- [8] F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology,” *IEEE Trans. VLSI Syst.*, vol. 27, no. 11, pp. 2629–2640, 2019.
- [9] C. X. Wolf, “PicoRV32 – a size-optimized RISC-V CPU,” <https://github.com/YosysHQ/picorv32>.
- [10] M. Liu *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” arXiv:2311.00176, 2023.
- [11] J. Blocklove *et al.*, “Automatically improving LLM-based Verilog generation using EDA tool feedback,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, 2025.
- [12] Y. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language model,” in *Proc. DAC*, 2024.
- [13] C.-T. Ho, H. Ren, and B. Khailany, “VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool,” in *Proc. AAAI*, 2025.
- [14] Y. Zhao *et al.*, “MAGE: A multi-agent engine for automated RTL code generation,” in *Proc. DAC*, 2025.
- [15] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [16] G. Lore, “Partial variable assignment leads to incorrect ordering/lack of circular logic detection,” Verilator issue #5829, <https://github.com/verilator/verilator/issues/5829>, 2025.
- [17] —, “Workaround for Verilator ordering issue in OpenPiton cache adapter,” CVA6 pull request #2809, <https://github.com/openhwgroup/cva6/pull/2809>, 2025.